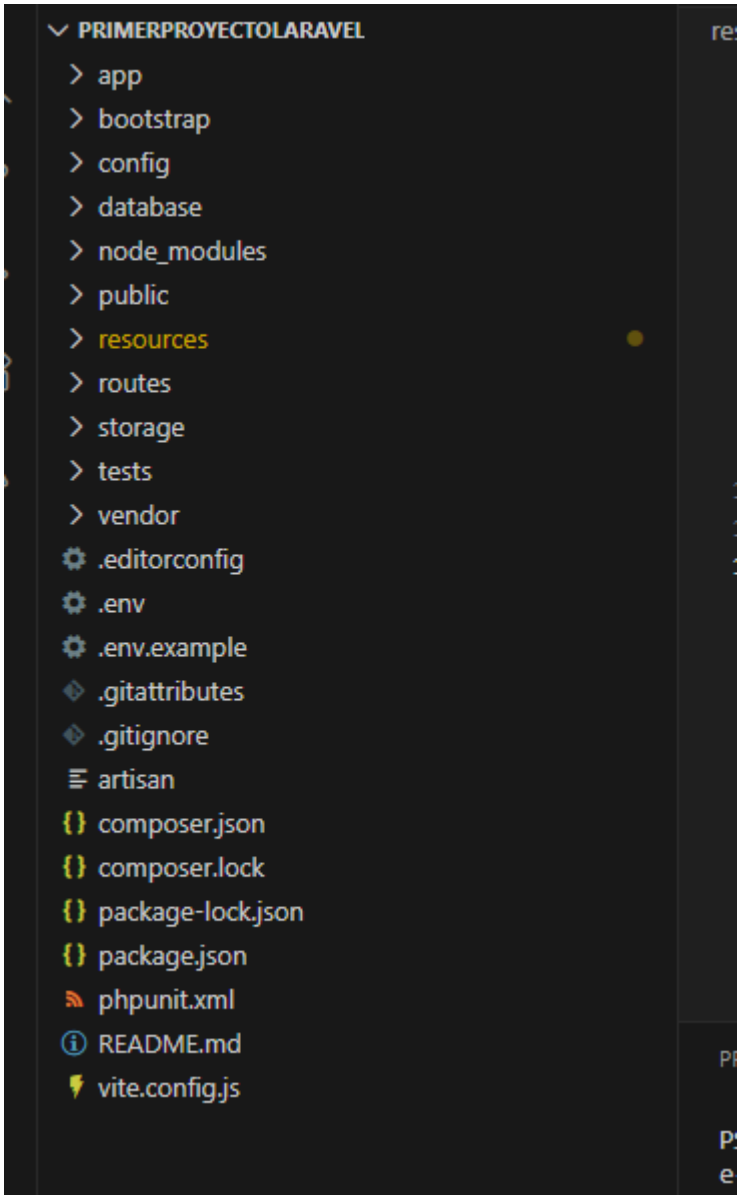
TAREA 10.1 Taller-parte1

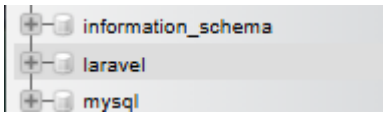
Comprobamos, en primer lugar, que la estructura de directorios es correcta tras la instalación de los softwares necesarios y la creación del proyecto:



Modifico el nombre de la base de datos en el archivo .env para configurar nuestra base de datos correctamente:

```
.env
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:Dfa69c5I7FbCIGcnId3V0zDw8XbRzh8qp7+xYbaAQsc=
APP_DEBUG=true
APP_URL=http://localhost

APP_LOCALE=en
APP_FALLBACK_LOCALE=en
APP_FAKER_LOCALE=en_US

APP_MAINTENANCE_DRIVER=file
# APP_MAINTENANCE_STORE=database

# PHP_CLI_SERVER_WORKERS=4

BCRYPT_ROUNDS=12

LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=
```

Hacemos un php artisan migrate en la terminal de Visual Studio Code y reviso en phpmyadmin si se ha creado la base de datos “laravel”.



Ahora utilizaremos las migraciones de Laravel (archivos que definen y gestionan la estructura de la base de datos).

En primer lugar, crearemos la tabla “libros” de esta manera con:

- php artisan make:migration create_libros_table y lo modificamos:

2026_02_04_155322_create_libros_table.php ● 2026_02_04_161402_create_categorias

merProyectoLaravel > database > migrations > 2026_02_04_155322_create_libros_table.php > class > up

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create(table: 'libros', callback: function (Blueprint $table):
            $table->id();
            $table->string(column: 'titulo');
            $table->integer(column: 'anyo_publicacion');
            $table->timestamps();

        });
    }
```

A continuación, tras añadir las columnas que queremos usar ejecutamos:

- php artisan migrate

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL powershell

```
2026_02_04_155322_create_libros_table ..... 121.50ms DONE
```

Compruebo que la tabla se ha creado correctamente:

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterm
1	id	bigint(20)		UNSIGNED	No	*Ninguna*
2	titulo	varchar(255)	utf8mb4_unicode_ci		No	*Ninguna*
3	anyo_publicacion	int(11)			No	*Ninguna*
4	created_at	timestamp			Sí	*NULL*
5	updated_at	timestamp			Sí	*NULL*
6	autor_id	bigint(20)		UNSIGNED	Sí	*NULL*

(Nota: el autor_id se crea en una práctica posterior).

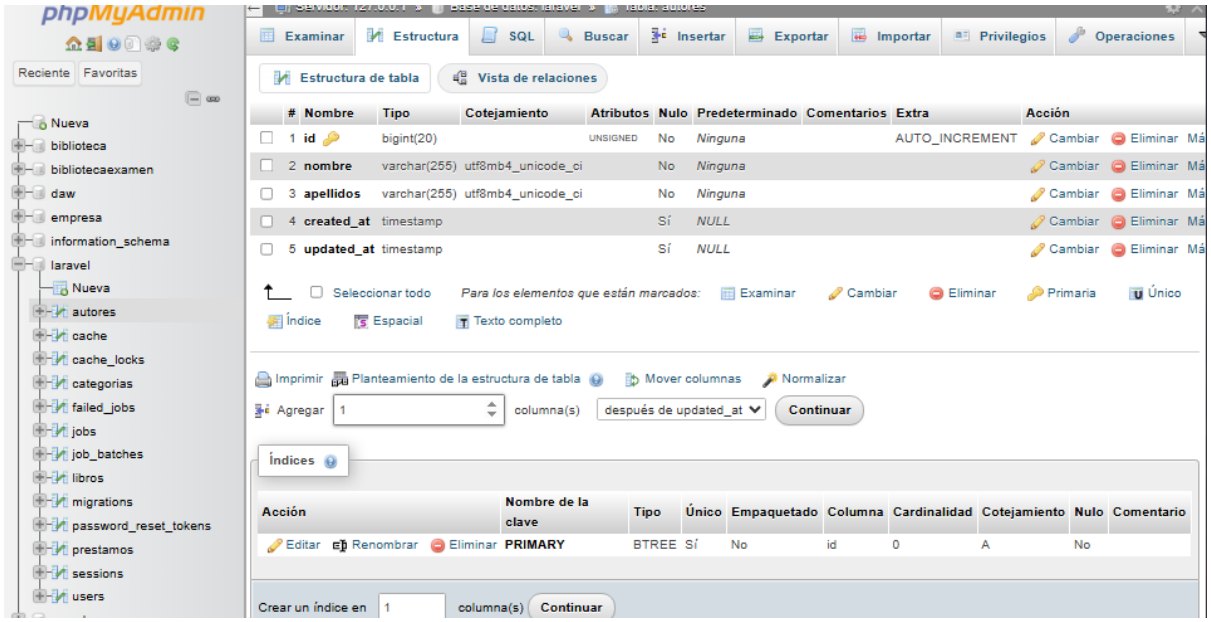
Repetimos este proceso con las otras tablas (autores, categorías y préstamos):

- php artisan make:migration create_autores_table:

database > migrations > 2026_02_04_161338_create_autores_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create(table: 'autores', callback: function (Blueprint $table): void {
            $table->id();
            $table->string(column: 'nombre');
            $table->string(column: 'apellidos');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists(table: 'autores');
    }
};
```

Resultados después de php artisan migrate:



- php artisan make:migration create_categorias_table:

)4_155322_create_libros_table.php | 2026_02_04_161402_create_categorias_table.php

PrimerProyectoLaravel > database > migrations > 2026_02_04_161402_create_categorias_table.php > class

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create(table: 'categorias', callback: function (Blueprint $tab
            $table->id();
            $table->string(column: 'nombre');
            $table->string(column: 'descripcion');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists(table: 'categorias');
    }
};
```

Resultado de la migración:

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comentarios	Extra	Acción
1	id	bigint(20)		UNSIGNED	No	*Ninguna*		AUTO_INCREMENT	Cambiar Eliminar Má
2	nombre	varchar(255)	utf8mb4_unicode_ci		No	*Ninguna*			Cambiar Eliminar Má
3	descripcion	varchar(255)	utf8mb4_unicode_ci		No	*Ninguna*			Cambiar Eliminar Má
4	created_at	timestamp			Sí	*NULL*			Cambiar Eliminar Má
5	updated_at	timestamp			Sí	*NULL*			Cambiar Eliminar Má

Seleccionar todo *Para los elementos que están marcados:* Examinar Cambiar Eliminar Primaria Único

- php artisan make:migration create_prestamos_table:

```
rias_table.php   prestamos.php   2026_02_04_161350_create_prestamos_table.php ●
abase > migrations > 2026_02_04_161350_create_prestamos_table.php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create(table: 'prestamos', callback: function (Blueprint $table)
        : void {
            $table->id();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists(table: 'prestamos');
    }
};
```

(Nota: Para este momento de la práctica no teníamos más columnas o campos para la tabla prestamos).

Migramos todas estas creaciones de tablas con:

- php artisan migrate:

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan migrate

  INFO  Running migrations.

  2026_02_04_161338_create_autores_table ..................................... 163.02ms DONE
  2026_02_04_161350_create_prestamos_table ................................... 130.58ms DONE
  2026_02_04_161402_create_categorias_table .................................. 160.19ms DONE

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel>
```

(Nota: Los resultados de las migraciones aparecen junto al código de la migración correspondiente)

Actualizo la tabla préstamos para añadir nuevas columnas con el comando:

con php artisan make:migration update_prestamos_table y la migro con php artisan migrate

2026_02_10_175717_update_prestamos_table.php × AutoresTableSeeder.php categoria

database > migrations > 2026_02_10_175717_update_prestamos_table.php

```php
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table(table: 'prestamos', callback: function (Blueprint $table)
        : void {
            $table->date(column: 'fecha_prestamos')->after(column: 'id');
            $table->date(column: 'fecha_limite');
            $table->date(column: 'fecha_devolucion');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        //
    }
};
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:migration update_prestamos_table

  INFO  Migration [C:\xampp\htdocs\formu\PrimerProyectoLaravel\database\migrations\2026_02_10_1757
17_update_prestamos_table.php] created successfully.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan migrate

  INFO  Running migrations.

  2026_02_10_175717_update_prestamos_table ..................................... 798.71ms DONE
```

Resultados de la migración:

Estructura de tabla | Vista de relaciones

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comenta
1	id	bigint(20)		UNSIGNED	No	Ninguna	
2	fecha_prestamos	date			No	Ninguna	
3	created_at	timestamp			Sí	NULL	
4	updated_at	timestamp			Sí	NULL	
5	libros_id	bigint(20)		UNSIGNED	Sí	NULL	
6	fecha_limite	date			No	Ninguna	
7	fecha_devolucion	date			No	Ninguna	

(Nota: libros_id es una clave foránea que haremos más adelante en la parte 2).

Una vez que hemos creado las tablas y definido sus columnas, procederemos a meter datos de prueba para la tabla a través de los seeders (Database>seeders).

Primero añadiremos datos a la tabla de libros usando el comando php artisan make:seeder LibrosTableSeeder.

```
2026_02_04_161402_create_categorias_table.php   LibrosTableSeeder.php   2026_02_04
PrimerProyectoLaravel > database > seeders > LibrosTableSeeder.php > PHP Intelephense > LibrosTableSeeder > run
1  <?php
2
3  namespace Database\Seeders;
4  use Illuminate\Support\Facades\DB;
5  use Illuminate\Database\Console\Seeds\WithoutModelEvents;
6  use Illuminate\Database\Seeder;
7
   0 references | 0 implementations
8  class LibrosTableSeeder extends Seeder
9  {
10     /**
11      * Run the database seeds.
12      */
       0 references | 0 overrides
13     public function run(): void
14     {
15         DB::table(table: "libros")->insert(values: [
16             "titulo"=>"El niño del pijama de rayas",
17             "anyo_publicacion"=>1991
18         ]);
19         DB::table(table: "libros")->insert(values: [
20             "titulo"=>"El ocho",
21             "anyo_publicacion"=>1990
22         ]);
23     }
24 }
25
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   powershell
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:seeder LibrosTableSeeder
```

Terminamos de insertar los datos del seeder en la base de datos usando php artisan db:seed --class=LibrosTableSeeder:

Mostrando filas 0 - 3 (total de 4, La consulta tardó 0,0003 segundos.)

```
SELECT * FROM `libros`
```

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Act

Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna

Opciones extra

	id	titulo	anyo_publicacion	created_at	updated_at
Editar Copiar Borrar	1	El niño del pijama de rayas	1991	NULL	NULL
Editar Copiar Borrar	2	El ocho	1990	NULL	NULL

Seleccionar todo *Para los elementos que están marcados:* Editar Copiar Borrar Exportar

Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna

Repito nuevamente el proceso para insertar datos en otras tablas:

- php artisan make:seeder AutoresTableSeeder

PrimerProyectoLaravel > database > seeders > AutoresTableSeeder.php > ...

```php
namespace Database\Seeders;
use Illuminate\Support\Facades\DB;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

0 references | 0 implementations
class AutoresTableSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    0 references | 0 overrides
    public function run(): void
    {
        DB::table(table: "autores")->insert(values: [
            "nombre"=>"John",
            "apellidos"=>"Boyne"
        ]);
        DB::table(table: "autores")->insert(values: [
            "nombre"=>"Katherine",
            "apellidos"=>"Neville"
        ]);
    }
}
```

```
INFO Seeding database.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan db:seed --class=LibrosTableSeeder

INFO Seeding database.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:seeder CategoriasTableSeeder

INFO Seeder [C:\xampp\htdocs\formu\PrimerProyectoLaravel\database\seeders\CategoriasTableSeeder.p
hp] created successfully.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:seeder AutoresTableSeeder
```

- Insertamos los datos de prueba del seeder con php artisan db:seed –-class=AutoresTableSeeder.

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan db:seed --class=AutoresTableSeeder

  INFO  Seeding database.
```

- Resultados en la base de datos:

Mostrando filas 0 - 1 (total de 2, La consulta tardó 0,0002 segundos.)

```
SELECT * FROM `autores`
```

Perfilando [Editar en línea] [E

Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla Orc

Opciones extra

←T→			id	nombre	apellidos	created_at	updated_at
Editar	Copiar	Borrar	1	John	Boyne	NULL	NULL
Editar	Copiar	Borrar	2	Katherine	Neville	NULL	NULL

- Creo el seeder de categorias con php artisan make:seeder CategoriasTableSeeder:

EXPLORER

FORMU
- PrimerProyectoLaravel
  - app
  - bootstrap
  - config
  - database
    - factories
      - UserFactory.php
    - migrations
      - 0001_01_01_0000...
      - 0001_01_01_0000...
      - 0001_01_01_0000...
      - 2026_02_04_1553...
      - 2026_02_04_1613...
      - 2026_02_04_1613...
      - 2026_02_04_1614...
    - seeders
      - AutoresTableSeed...
      - CategoriasTableSe...
      - DatabaseSeeder.p...
      - LibrosTableSeeder...
    - .gitignore
    - database.sqlite
  - public
  - resources
  - routes
  - storage
  - tests

AutoresTableSeeder.php | CategoriasTableSeeder.php | 2026_02_04_161338_create_

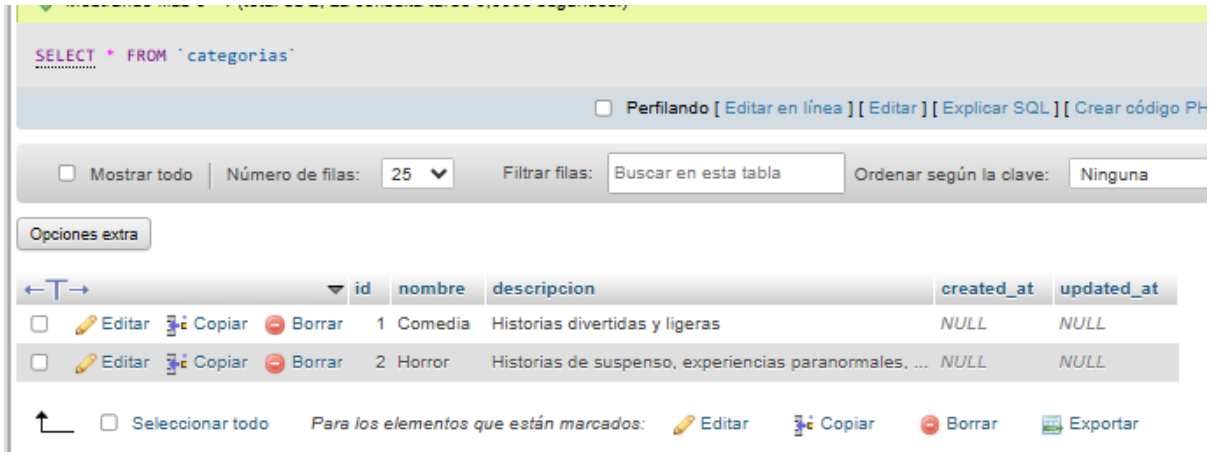
PrimerProyectoLaravel > database > seeders > CategoriasTableSeeder.php > PHP Intelephense > CategoriasTableSeede

```
6   use Illuminate\Database\Seeder;
7
    0 references | 0 implementations
8   class CategoriasTableSeeder extends Seeder
9   {
10      /**
11       * Run the database seeds.
12       */
        0 references | 0 overrides
13      public function run(): void
14      {
15          DB::table(table: "categorias")->insert(values: [
16              "nombre"=>"Comedia",
17              "descripcion"=>"Historias divertidas y ligeras"
18          ]);
19          DB::table(table: "categorias")->insert(values: [
20              "nombre"=>"Horror",
21              "descripcion"=>"Historias de suspenso, experiencias paranormales, etc..."
22          ]);
23      }
24  }
25
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
  INFO  Seeding database.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:seeder CategoriasTableSeeder

  INFO  Seeder [C:\xampp\htdocs\formu\PrimerProyectoLaravel\database\seeders\CategoriasTableSeeder.p
hp] created successfully.
```

- Insertamos los datos del seeder con php artisan db:seed –-class=CategoriasTableSeeder.



- Creo el seeder para PrestamosTableSeeder con php artisan make:seeder PrestamosTableSeeder:

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:seeder PrestamosTableSeeder

  INFO  Seeder [C:\xampp\htdocs\formu\PrimerProyectoLaravel\database\seeders\PrestamosTableSeeder.
php] created successfully.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel>
```

```
7_update_prestamos_table.php   PrestamosTableSeeder.php ×   LibrosTableSeeder.php
database > seeders > PrestamosTableSeeder.php > ...
1   <?php
2
3   namespace Database\Seeders;
4
5   use Illuminate\Database\Console\Seeds\WithoutModelEvents;
6   use Illuminate\Database\Seeder;
7   use Illuminate\Support\Facades\DB;
8
    0 references | 0 implementations
9   class PrestamosTableSeeder extends Seeder
10  {
11      /**
12       * Run the database seeds.
13       */
        0 references | 0 overrides
14      public function run(): void
15      {
16          DB::table(table: 'prestamos')->insert(values: [
17              'fecha_prestamos' => '2025-12-12',
18              'fecha_limite' => '2026-02-12',
19              'fecha_devolucion' => '2026-01-05'
20          ]);
21      }
22  }
23
```

Ejecuto el seeder y veo si se han creado los datos

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan db:seed --class=PrestamosTableSeeder

  INFO  Seeding database.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel>
```

←T→	id	fecha_prestamos	created_at	updated_at	libros_id	fecha_limite	fecha_devolucion
Editar Copiar Borrar	1	2025-12-12	NULL	NULL	NULL	2026-02-12	2026-01-05

TAREA 10.1 Taller-parte2

En esta segunda parte del taller, nos vamos a centrar en la creación de relaciones entre las tablas. Veremos cómo crear una relación 1:N y N:M..

- Creamos una migración para alterar la tabla de libros y meter la clave foránea.
    - php artisan make:migration alter_libros_table.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table(table: 'libros', callback: function (Blueprint $table): void {
            $table->unsignedBigInteger(column: 'autor_id')->nullable();
            $table->foreign(columns: 'autor_id')->references(columns: 'id')->on(table: 'autores')
            ->onDelete(action: 'set null');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        //
    }
};
```

- Confirmamos la creación de esta clave foránea con php artisan migrate:

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan migrate

  INFO  Running migrations.

  2026_02_06_145301_alter_libros_table ..................................... 387.32ms DONE
```

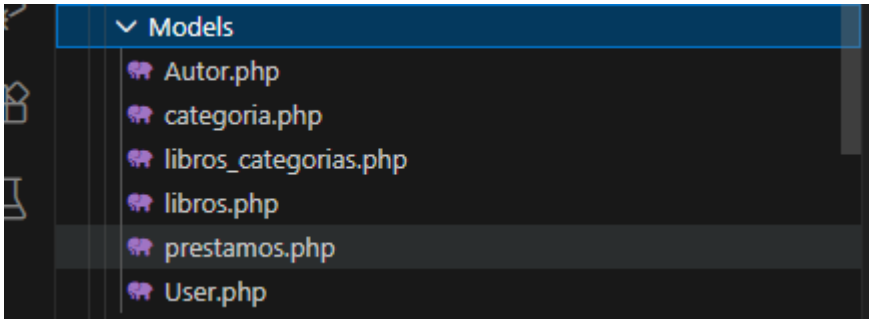
Para la creación de estas relaciones, vamos a necesitar un modelo (archivo que define la clase que contiene las relaciones de una tabla en concreto con otras tablas a través del modelo de estas últimas) por cada tabla que busquemos relacionar. En el caso de la relación N:M (libros y categorías) tendremos que crear un último modelo para la tabla intermedia.

- Creamos todos los modelos que requerimos para nuestras relaciones con el comando:
    - php artisan make:model nombreModelo

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:model libros

  INFO  Model [C:\xampp\htdocs\formu\PrimerProyectoLaravel\app\Models\libros.php] created success
fully.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:model Autor

  INFO  Model [C:\xampp\htdocs\formu\PrimerProyectoLaravel\app\Models\Autor.php] created successf
ully.
```

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:model prestamos

  INFO  Model [C:\xampp\htdocs\formu\PrimerProyectoLaravel\app\Models\prestamos.php] created succ
essfully.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:model categoria

  INFO  Model [C:\xampp\htdocs\formu\PrimerProyectoLaravel\app\Models\categoria.php] created succ
essfully.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:model libros_categorias

  INFO  Model [C:\xampp\htdocs\formu\PrimerProyectoLaravel\app\Models\libros_categorias.php] crea
ted successfully.
```

- Resultado en la carpeta app>Models:



- Código del modelo Autor.php:

app > Models > Autor.php > Autor > libros()

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;


1 reference | 0 implementations
class Autor extends Model
{
    use HasFactory;
    0 references
    protected $table = "autores";
    0 references
    protected $primaryKey = 'id';
    0 references | 0 overrides
    public function libros(): HasMany{
        return $this->hasMany(related: libros::class);
    }
}

```

- Código del modelo categoria.php:

libros.php | Autor.php | categoria.php ×

app > Models > categoria.php > ...

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

2 references | 0 implementations
class categoria extends Model
{
    0 references | 0 overrides
    public function libros(): BelongsToMany{
        return $this->belongsToMany(related: libros::class,
        table: 'libros_categorias', foreignPivotKey: 'categoria_id',
        relatedPiv…'libros_id');
    }
}

```

(Nota: En las relaciones de Laravel hay que tener mucho cuidado con los nombres ya que Laravel utiliza una sintaxis por defecto que no tiene que coincidir necesariamente con los nombres que hemos definido anteriormente. Para evitar problemas se añaden parámetros adicionales al método belongsTo/ToMany además del modelo con el que está relacionado. El segundo parámetro hace referencia en este caso al nombre de la tabla intermedia con la que se relaciona el modelo con el que trabajo, el tercer parámetro es la clave de mi modelo que se relacionará y el cuarto parámetro hace referencia a la clave del modelo contrario con el que me relaciono. Estos parámetros se utilizarán en algunos modelos posteriores).

- Código del modelo libros.php:

(Nota: Como puede verse, libros tiene relaciones con autor, préstamos y categorias, definidas a través de funciones de la clase que se relacionan a los modelos que crearemos más adelante).

app > Models > libros.php > libros > categorias()

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

9 references | 0 implementations
class libros extends Model
{
    use HasFactory;
    0 references
    protected $table = "libros";
    0 references
    protected $primaryKey = 'id';
    0 references | 0 overrides
    public function autor(): BelongsTo{
        return $this->belongsTo(related: Autor::class);
    }

    0 references | 0 overrides
    public function prestamos(): HasMany{
        return $this->hasMany(related: prestamos::class);
    }

    0 references | 0 overrides
    public function categorias(): BelongsToMany{
        return $this->belongsToMany(related: categoria::class,
        table: 'libros_categorias', foreignPivotKey: 'libro_id');
    }
}
```

- Creo la clave foránea en la tabla de préstamos usando php artisan make:migration alter_prestamos_table:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table(table: 'prestamos', callback: function (Blueprint $table): void {
            $table->unsignedBigInteger(column: 'libros_id')->nullable();
            $table->foreign(columns: 'libros_id')->references(columns: 'id')->on(table: 'libros')
            ->onDelete(action: 'set null'); });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        //
    }
};
```

- Creo la tabla intermedia para la relación N:M entre libros y categorias con el comando:
    - php artisan make:migration create_libros_categoria.

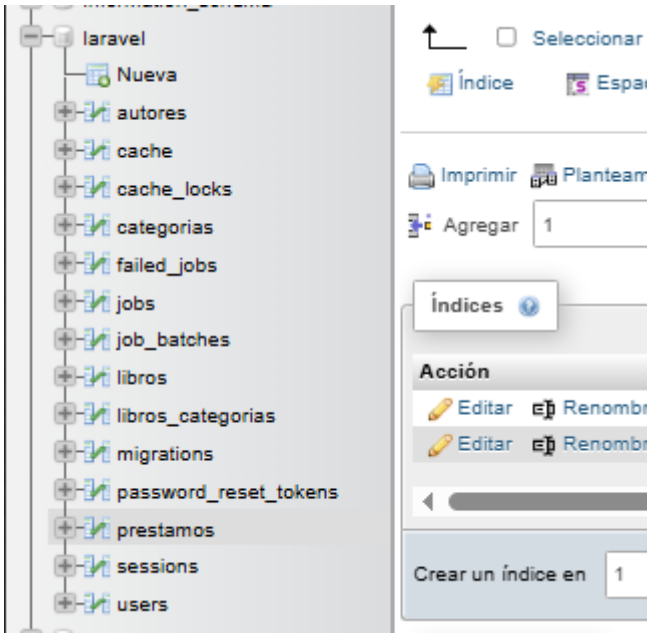
```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:migration create_libros_categoria
s_table

  INFO  Migration [C:\xampp\htdocs\formu\PrimerProyectoLaravel\database\migrations\2026_02_06_152
720_create_libros_categorias_table.php] created successfully.
```

2026_02_06_152720_create_libros_categorias_table.php | categoria.php | libros_categorias.php | api.php

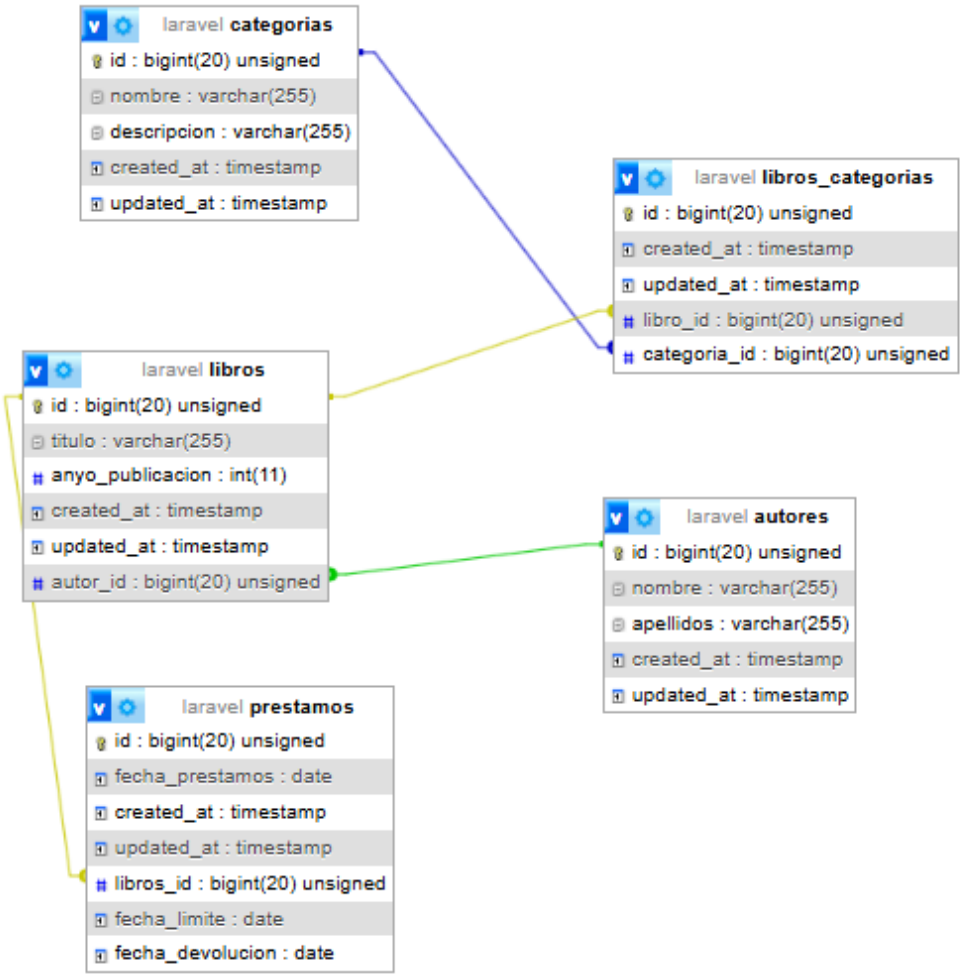
PrimerProyectoLaravel > database > migrations > 2026_02_06_152720_create_libros_categorias_table.php

```php
return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create(table: 'libros_categorias', callback: function (Blueprint $table): void {
            $table->id();
            $table->timestamps();

            $table->unsignedBigInteger(column: 'libro_id');
            $table->unsignedBigInteger(column: 'categoria_id');

            $table->foreign(columns: 'libro_id')->references(columns: 'id')->on(table: 'libros');
            $table->foreign(columns: 'categoria_id')->references(columns: 'id')->on
            (table: 'categorias');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists(table: 'libros_categorias');
    }
};
```

- Confirmo todos los cambios realizados con:
  - php artisan migrate



He aquí los resultados de estas relaciones:



- Creo en la carpeta de routes el archivo web.php para asociar la URL a una función específica del controlador ApiController (ENDPOINTs):

```php
<?php
    namespace App\Http\Controllers\Api;
    use Illuminate\Support\Facades\Route;
    use App\Http\Controllers\ApiController;
    Route::get(uri: '/', action: function (): View {
        return view(view: 'welcome'); });
    Route::get(uri: '/test', action: function (): string {
        return 'API OK'; });
    Route::get(uri: '/indice', action: [ApiController::class, 'indice']);
    Route::get(uri: '/libros', action: [ApiController::class, 'index']);
```

- Creo el controlador utilizado en web.php con el comando:
    - php artisan make:controller ApiController

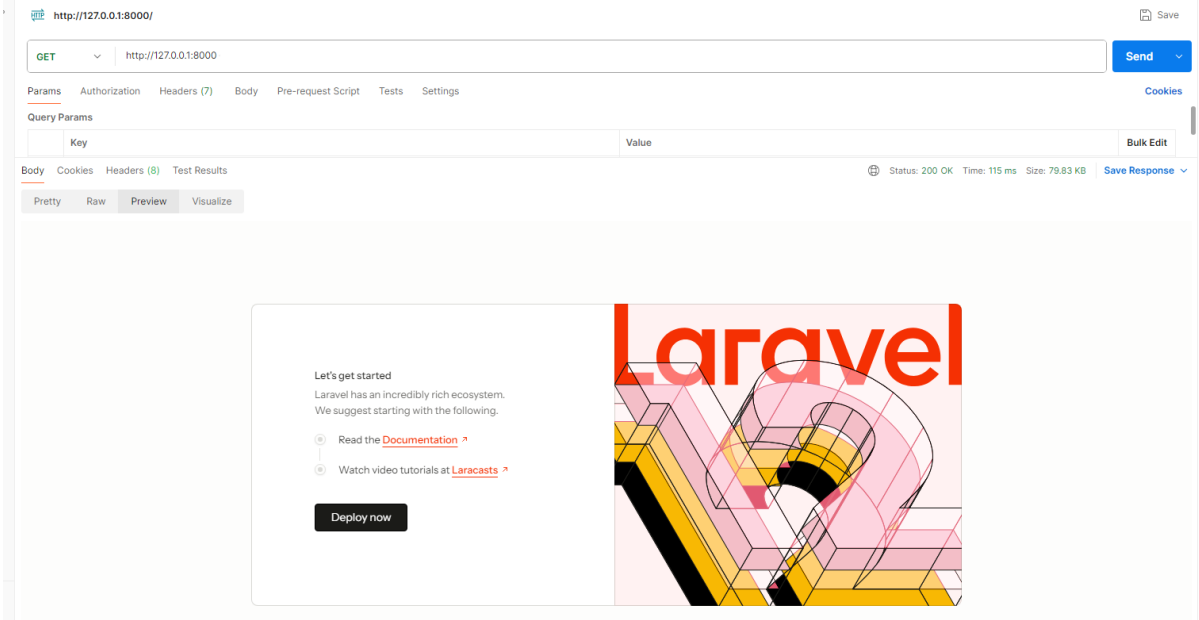
```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan make:controller ApiController

  INFO  Controller [C:\xampp\htdocs\formu\PrimerProyectoLaravel\app\Http\Controllers\ApiController.php] created successfully.
```

- Escribimos el siguiente código en el controlador creado (se encontrará en la ruta App>Http>Controllers) que contiene las funciones:
    - Index: Devuelve en formato JSON los libros con información del autor.
    - Indice: Simplemente muestra un mensaje de OK.

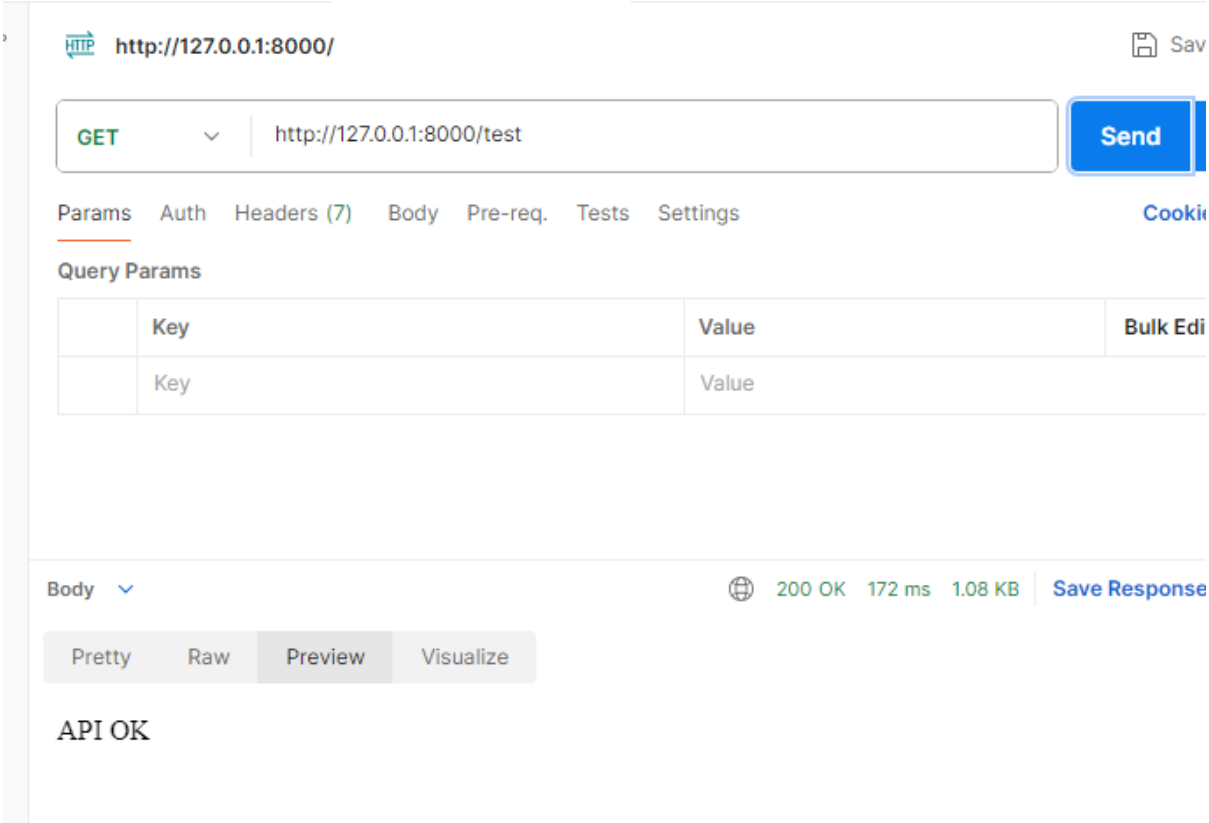
```php
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Controllers\Controller;
use App\Models\libros;
5 references | 0 implementations
class ApiController extends Controller
{
    2 references | 0 overrides
    public function index(): JsonResponse
    {
        $libros = libros::with(relations: 'autor')->get();
        return response()->json(data: $libros);
    }
    1 reference | 0 overrides
    public function indice(): JsonResponse
    {
        return response()->json(data: ['message' => 'OK']);
    }
}
```

- Descargamos y abrimos postman para realizar consultas en la API:

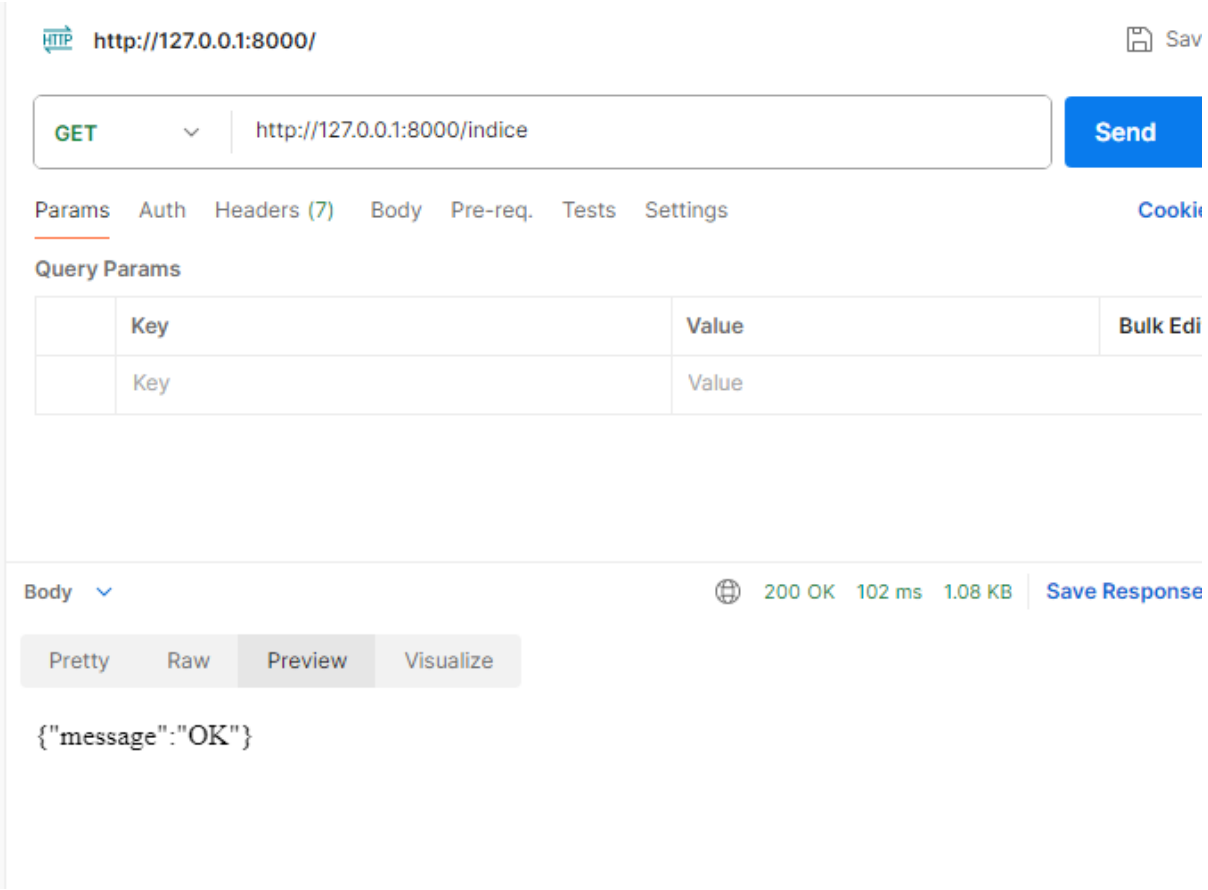
GET http://127.0.0.1:8000



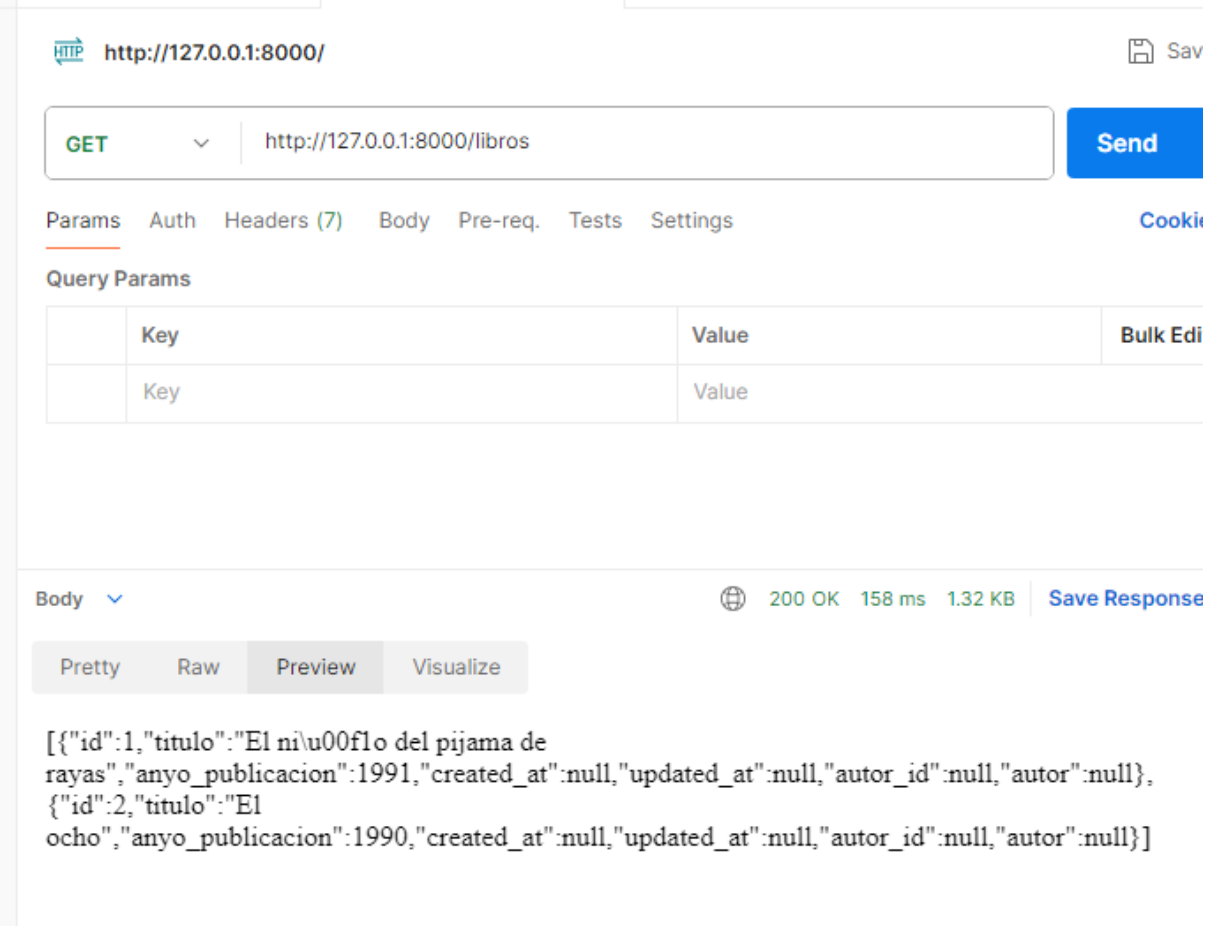
GET http://127.0.0.1:8000/test



GET http://127.0.0.1:8000/indice



GET http://127.0.0.1:8000/libros



Como hemos añadido estos libros antes de crear la clave foránea no hay información de los autores, pero si añadiera un libro después de ejecutar el alta de un libro (más adelante) saldrá la información del autor correspondiente al rellenar el nuevo campo de autor_id:

```
[
    {
        "id": 1,
        "titulo": "El niño del pijama de rayas",
        "anyo_publicacion": 1991,
        "created_at": null,
        "updated_at": null,
        "autor_id": null,
        "autor": null
    },
    {
        "id": 2,
        "titulo": "El ocho",
        "anyo_publicacion": 1990,
        "created_at": null,
        "updated_at": null,
        "autor_id": null,
        "autor": null
    },
    {
        "id": 4,
        "titulo": "El libro de la vida",
        "anyo_publicacion": 2026,
        "created_at": "2026-02-10T18:50:01.000000Z",
        "updated_at": "2026-02-10T18:50:01.000000Z",
        "autor_id": 2,
        "autor": {
            "id": 2,
            "nombre": "Katherine",
            "apellidos": "Neville",
            "created_at": null,
            "updated_at": null
        }
```


- Deseamos crear un nuevo ENDPOINT en el fichero web.php y ls función con las que está asociada que tenga la función de añadir libros. Para ello:
    - Añadimos un nuevo ENDPOINT (url) al fichero web.php.

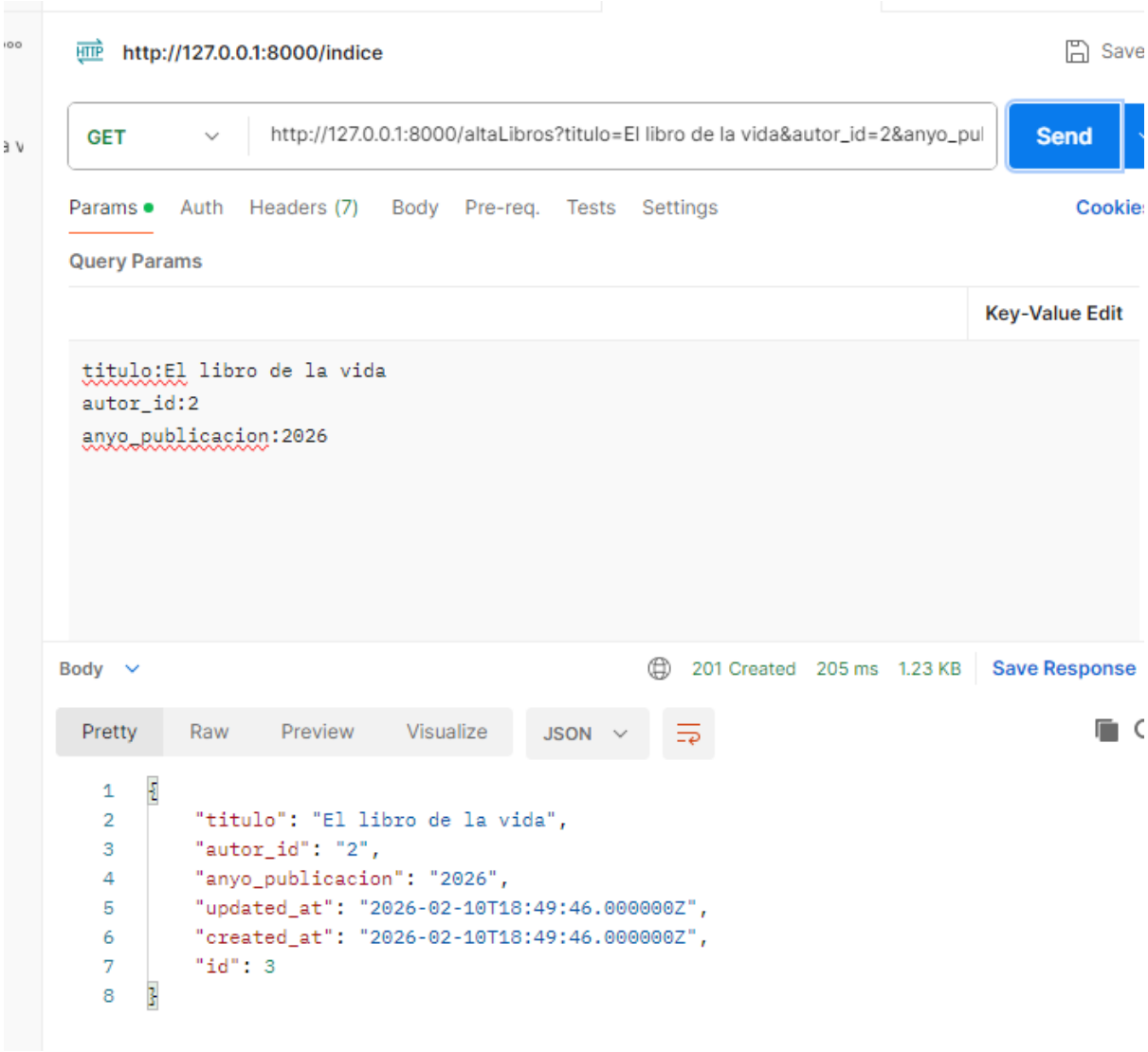
```
Route::get(uri: '/altaLibros', action: [ApiController::class, 'altaLibros']);
```

    - Creamos la función altaLibros en nuestro ApiController:

```
1 reference | 0 overrides
public function altaLibros(Request $request): JsonResponse
{
    try {
        $request->validate(rules: ['titulo' => 'required|string', 'autor_id' => 'required|integer',
        'anyo_publicacion' => 'required|integer',]);
        $libro = new libros();
        $libro->titulo = $request->input(key: 'titulo');
        $libro->autor_id = $request->input(key: 'autor_id');
        $libro->anyo_publicacion = $request->input(key: 'anyo_publicacion');
        $libro->save();
        return response()->json(data: $libro, status: 201);
    } catch (\Exception $e) {
        return response()->json(data: ['error' => $e->getMessage()], status: 500);
    }
}

}
```

Probamos el ENDPOINT:

GET: http://127.0.0.1:8000/altaLibros



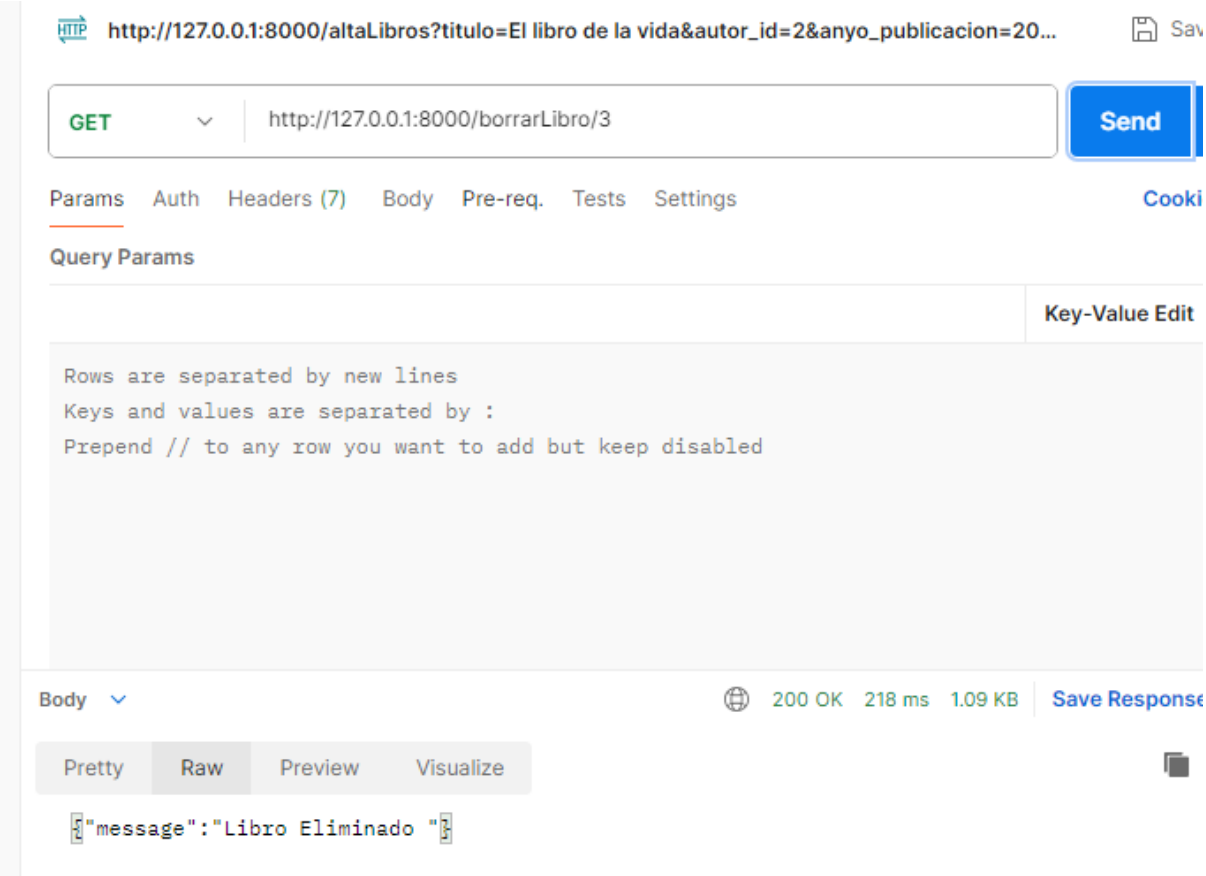
- Ahora queremos añadir un nuevo ENDPOINT (url) para borrar un libro y para ello:

- Añadimos la ruta en web.php y añadimos la función eliminar en nuestro ApiController (en esta función necesitaremos recibir el id):

```
Route::get(uri: '/borrarLibro/{id}', action: [ApiController::class, 'eliminar']);
```

```
1 reference | 0 overrides
public function eliminar($id): JsonResponse
{
    $libro = libros::find(id: $id);
    if (!$libro) {
        return response()->json(data: ['message' => 'El libro no existe'], status: 404);
    } else {
        $libro->delete();
    }
    return response()->json(data: ['message' => 'Libro Eliminado '], status: 200);
}
```

GET : http://127.0.0.1:8000/borrarLibro/3 en POSTMAN



- Y en nuestra base de datos ha desaparecido el libro número 3:

Opciones extra

	id	titulo	anyo_publicacion	created_at	updated_at	autor_id
Editar Copiar Borrar	1	El niño del pijama de rayas	1991	NULL	NULL	NULL
Editar Copiar Borrar	2	El ocho	1990	NULL	NULL	NULL
Editar Copiar Borrar	4	El libro de la vida	2026	2026-02-10 18:50:01	2026-02-10 18:50:01	2

Seleccionar todo *Para los elementos que están marcados:* Editar Copiar Borrar Exportar

- Y si pruebo con un libro que no existe:

GET http://127.0.0.1:8000/borrarLibro/54893273405737540 Send

Params Auth Headers (7) Body Pre-req. Tests Settings Cookie

Query Params

Key-Value Edit

```
Rows are separated by new lines
Keys and values are separated by :
Prepend // to any row you want to add but keep disabled
```

Body 404 Not Found 630 ms 1.1 KB Save Response

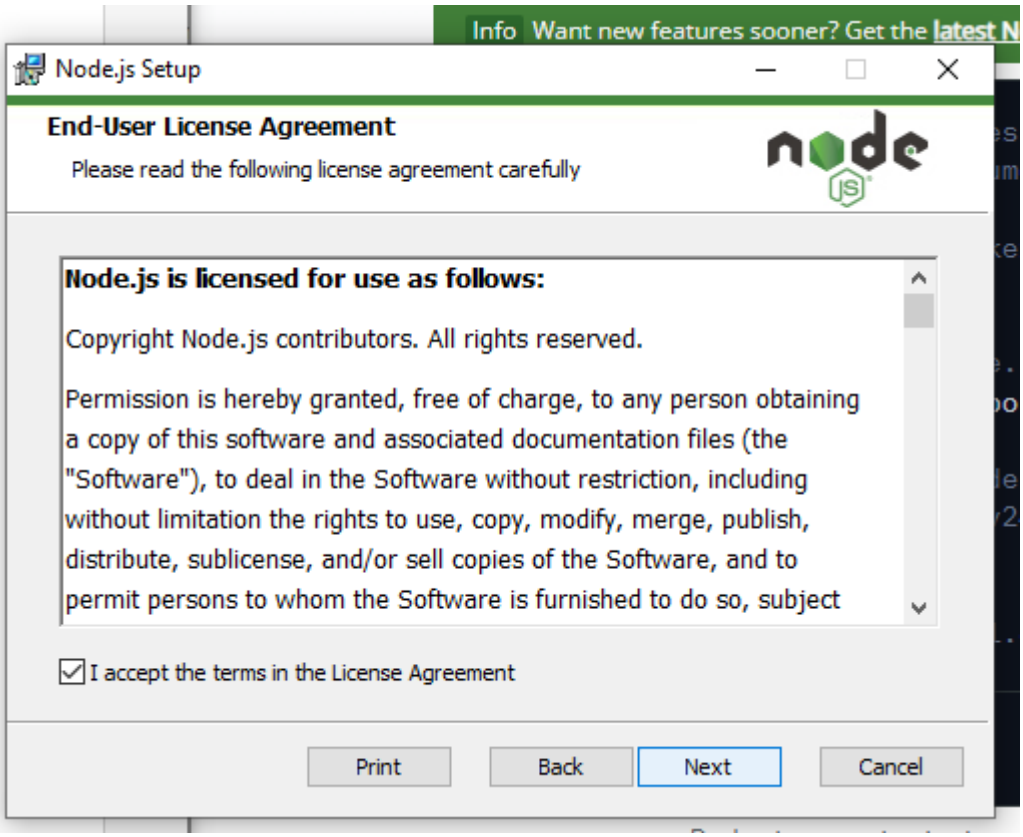
Pretty Raw Preview Visualize

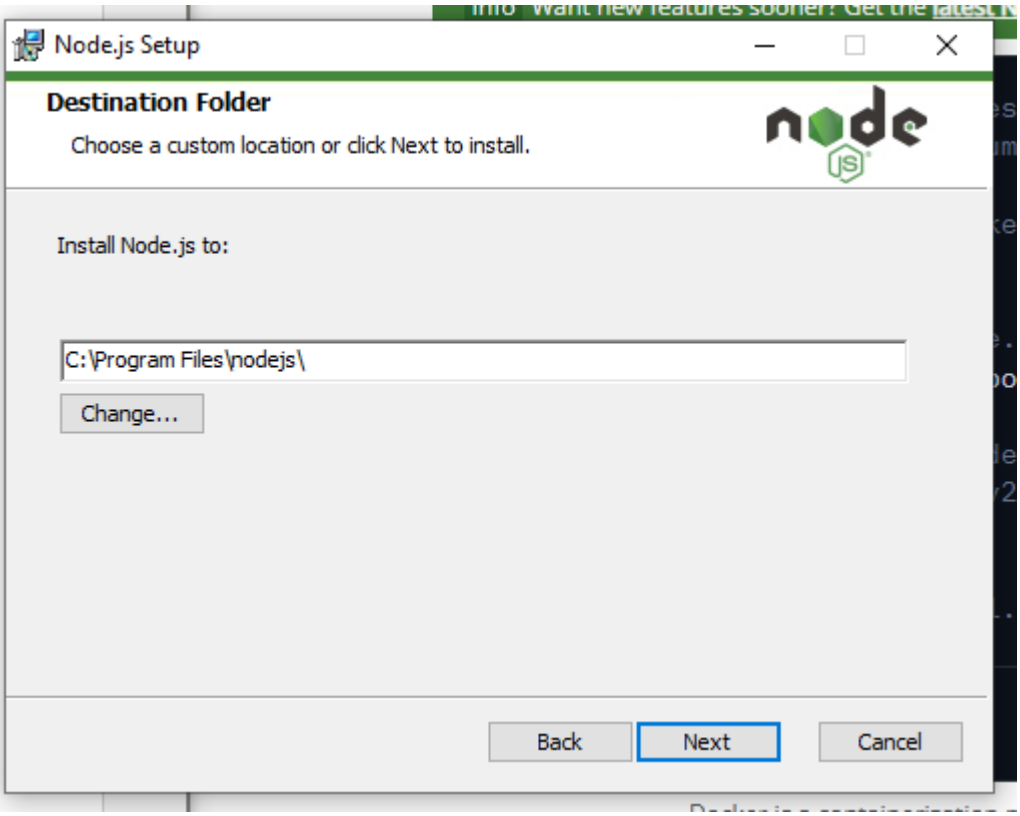
```
{"message":"El libro no existe"}
```

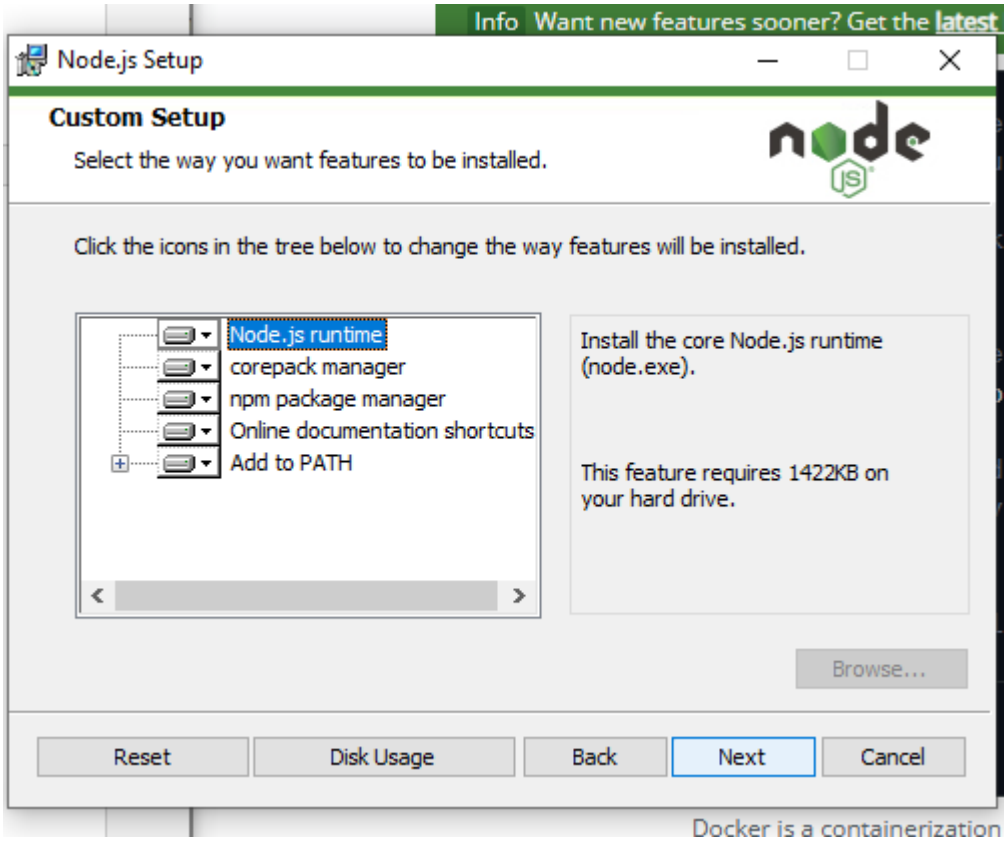
TAREA 10.1 Taller-parte3

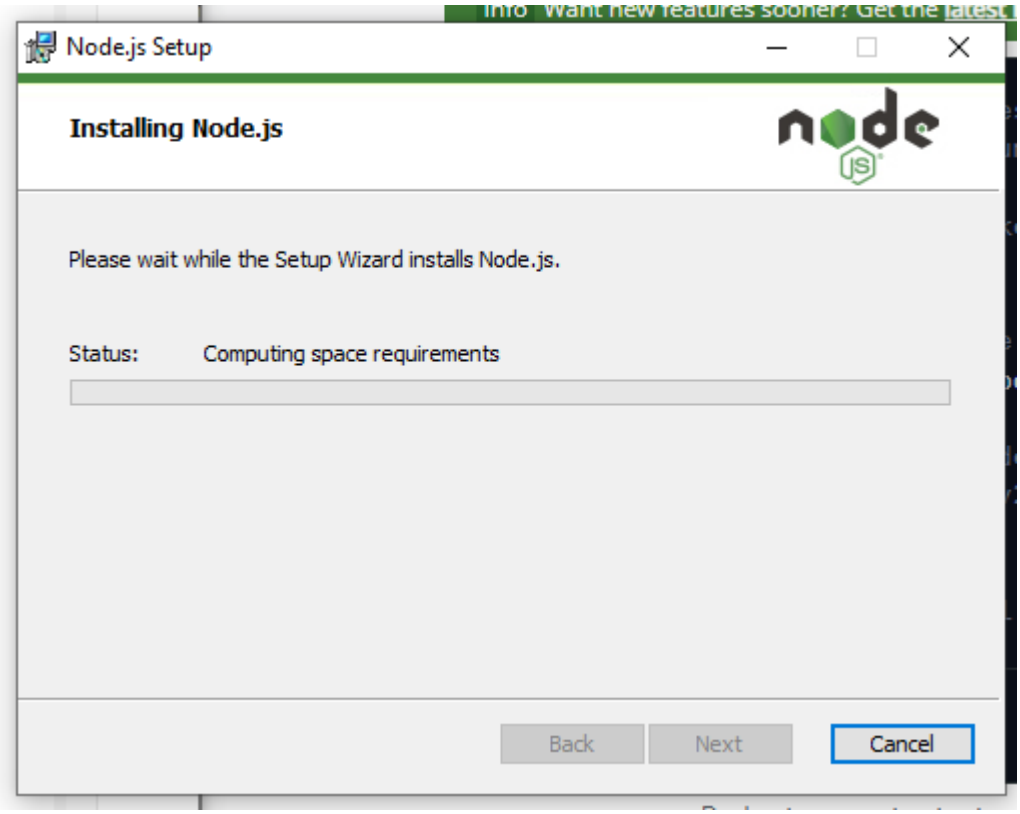
- Instalamos node.js a través de la URL nodejs.org/es/download

- Entramos y le damos a next y hacemos el procedimiento habitual e intuitivo:




Node.js Setup
Destination Folder
Choose a custom location or click Next to install.
Install Node.js to:
C:\Program Files\nodejs\
Change...
Back
Next
Cancel


Info Want new features sooner? Get the latest
Node.js Setup
Custom Setup
Select the way you want features to be installed.
Click the icons in the tree below to change the way features will be installed.
Node.js runtime
corepack manager
npm package manager
Online documentation shortcuts
Add to PATH
Install the core Node.js runtime (node.exe).
This feature requires 1422KB on your hard drive.
Browse...
Reset
Disk Usage
Back
Next
Cancel
Docker is a containerization



- Una vez instalado el Node.js vamos a nuestra terminal de VSC y ejecutamos los siguientes comandos para comprobar si este se ha instalado correctamente:
  - node -v
  - npm –v

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> node -v
v24.13.1
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> npm -v
npm : No se puede cargar el archivo C:\Program Files\nodejs\npm.ps1 porque la ejecución de
scripts está deshabilitada en este sistema. Para obtener más información, consulta el tema
about_Execution_Policies en https:/go.microsoft.com/fwlink/?LinkID=135170.
En línea: 1 Carácter: 1
+ npm -v
+ ~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel>
```

(Nota: Nuestro equipo de clase por seguridad tiene bloqueada la ejecución de Scripts así que tenemos que desbloquearlo con el siguiente comando en un PowerShell con permisos de administrador: Set-ExecutionPolicy RemoteSigned -Scope CurrentUser)

```
Administrador: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Prueba la nueva tecnología PowerShell multiplataforma https://aka.ms/pscore6

PS C:\WINDOWS\system32> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

Cambio de directiva de ejecución
La directiva de ejecución te ayuda a protegerte de scripts en los que no confías. Si cambias dicha directiva, podrías exponerte a los riesgos de
seguridad descritos en el tema de la Ayuda about_Execution_Policies en https:/go.microsoft.com/fwlink/?LinkID=135170. ¿Quieres cambiar la directiva
 de ejecución?
[S] Sí  [O] Sí a todo  [N] No  [T] No a todo  [U] Suspender  [?] Ayuda (el valor predeterminado es "N"): S
PS C:\WINDOWS\system32>
```

- Cierro y vuelvo a abrir Visual Studio Code y npm debería funcionar sin problemas.

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> npm -v
11.8.0
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel>
```

- Instalamos Vue a través de npm con el comando:
  - npm install vue –-save-dev

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> npm install vue --save-dev

added 100 packages, and audited 101 packages in 1m

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel>
```

- Instalamos también el plugin de vitejs para poder utilizar los componentes que veremos más adelante con el comando:
  - npm install @vitejs/plugin-vue

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> npm install @vitejs/plugin-vue

added 2 packages, and audited 103 packages in 3s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel>
```

- Configuramos nuestro archivo vite.config.js:

vite.config.js × | app.js | welcome.blade.php

vite.config.js > ...

```
import { defineConfig } from "vite";
import laravel from "laravel-vite-plugin";
import vue from "@vitejs/plugin-vue";
import path from "path";
export default defineConfig({
    plugins: [
        vue(),
        laravel({
            input: ["resources/css/app.css", "resources/js/app.js"],
            refresh: true,
        }),
    ],
});
```

- Configuramos el archivo resources/js/app.js que tendrá importados los componentes de ListaLibros (comentado por ahora) y HolaMundo:

PrimerProyectoLaravel

vite.config.js | app.js × | welcome.blade.php

resources > js > app.js

```
import "./bootstrap";
import { createApp } from "vue";

//import ListaLibros  from './componentes/ListaLibros.vue';
import HolaMundo from './componentes/HolaMundo.vue';

createApp(HolaMundo).mount("#app");
//createApp(ListaLibros).mount("#app");
```

- Ajustamos nuestra página de bienvenida a través del fichero welcome.blade.php en resource>views:

```
vite.config.js   app.js   welcome.blade.php
resources > views > welcome.blade.php
1  <!DOCTYPE html>
2  <html lang="{{ str_replace(search: '_', replace: '-', subject: app()->getLocale())
3
4  <head>
5      <meta charset="utf-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1">
7      <title>LARAVEL CON VUE</title> @vite(entrypoints: 'resources/css/app.css')
8  </head>
9
10 <body>
11     <div id="app"> </div> @vite(entrypoints: 'resources/js/app.js')
12 </body>
13
14 </html>
```

- Creamos nuestro primer componente al cual llamaremos HolaMundo.vue en resources>js>componentes:

(Nota: La carpeta componentes ha sido creada de forma manual dentro de la carpeta resources>js:

```
vite.config.js   app.js   welcome.blade.php   HolaMundo.vue
resources > js > componentes > HolaMundo.vue
1  <template>
2      <div>
3          <p>Hola Mundo !! </p>
4      </div>
5  </template>
```

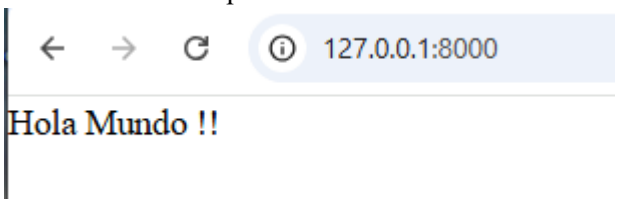
- Para poder visualizar los componentes necesitos arrancar los servidores de laravel y npm con
  - php artisan serve

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan serve

  INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server
```

  - npm run dev

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> npm run dev

> dev
> vite


  VITE v7.3.1  ready in 1014 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help

  LARAVEL v12.49.0  plugin v2.1.0

  ➜  APP_URL: http://localhost
```

- Accedemos a nuestro navegador para comprobar el componente HolaMundo.vue:
  - http://127.0.0.1:8000



- Creamos un nuevo componente para el listado de libros (ListaLibros.vue) y para ello debemos:
  - Comentar el componente de HolaMundo.vue en app.js.
  - Creamos el componente ListaLibros.vue en la carpeta de componentes.

```
resources > js > componentes > ListaLibros.vue
<template>
  <div>
    </ul>
  </div>
</template>

<script>
export default {
  data() {
    return {
      libros: [],
    };
  },
  mounted() {
    // Realiza una solicitud GET a la API de Laravel para obtener la lista de libros
    fetch("http://127.0.0.1:8000/api/libros")
      .then((response) => response.json())
      .then((data) => {
        this.libros = data;
      });
  },
};
</script>
```

```
fig.js   app.js   welcome.blade.php   HolaMundo.vue   List
esources > js > app.js
import "./bootstrap";
import { createApp } from "vue";

import ListaLibros  from './componentes/ListaLibros.vue';
//import HolaMundo from './componentes/HolaMundo.vue';

//createApp(HolaMundo).mount("#app");
createApp(ListaLibros).mount("#app");
```

- Veo el contenido del nuevo componente en el navegador, pero antes limpio el caché de las rutas para que no ocurran errores y compruebo que libros aparece como una ruta legítima a través de los comandos:
    - php artisan route:clear
    - php artisan route:list

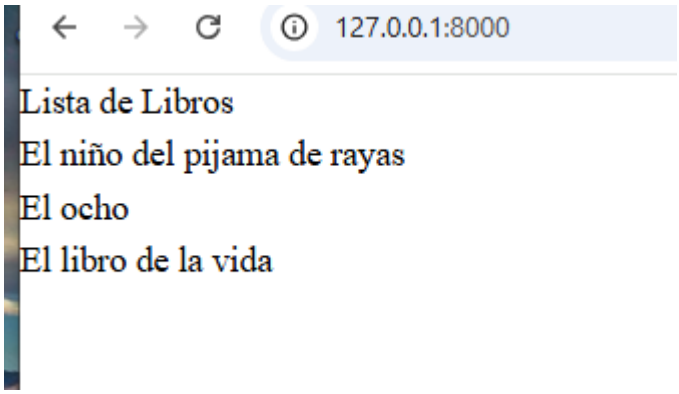
```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan route:clear

  INFO  Route cache cleared successfully.

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> php artisan route:list

  GET|HEAD        / ..............................................
  GET|HEAD        altaLibros ............................ ApiController@altaLibros
  GET|HEAD        borrarLibro/{id} ......................... ApiController@eliminar
  GET|HEAD        indice ................................... ApiController@indice
  GET|HEAD        libros ................................... ApiController@index
  GET|HEAD        storage/{path} ................................ storage.local
  GET|HEAD        test ...........................................
  GET|HEAD        up .............................................

                                                   Showing [8] routes

PS C:\xampp\htdocs\formu\PrimerProyectoLaravel>
```

(Nota: Como mis rutas no son api/... debería cambiar el fetch en el componente ListaLibros.vue, de lo contrario no encontrará la ruta porque no existe /api/ListaLibros.vue):

resources > js > componentes > ListaLibros.vue

```
<template>
  <div>
    <h2>Lista de Libros</h2>
    <ul>
      <li v-for="libro in libros" :key="libro.id">
        {{ libro.titulo }}
      </li>
    </ul>
  </div>
</template>

<script>
export default {
  data() {
    return {
      libros: [],
    };
  },
  mounted() {
    // Realiza una solicitud GET a la API de Laravel para obtener la lista de libros
    fetch("/libros")
      .then((response) => response.json())
      .then((data) => {
        this.libros = data;
      });
  },
};
</script>
```

- Busco en el navegador y:



Para finalizar con este taller instalaremos Vue Router que permite navegación en aplicaciones web:

- Instalamos con: npm install vue-router

```
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel> npm install vue-router

added 39 packages, and audited 142 packages in 7s

36 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm notice
npm notice New minor version of npm available! 11.8.0 -> 11.9.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.9.0
npm notice To update run: npm install -g npm@11.9.0
npm notice
PS C:\xampp\htdocs\formu\PrimerProyectoLaravel>
```

- Configuramos el archivo resources/js/router.js

```
resources > js > router.js > default
import { createRouter, createWebHistory } from 'vue-router';
import ListaLibros from './componentes/ListaLibros.vue';

const routes = [
  {
    path: '/libros',
    component: ListaLibros,
  }
];

const router = createRouter({
  history: createWebHistory(),
  routes,
});

export default router;
```

- Adaptamos el app.js:

```
resources > js > app.js
import "./bootstrap";
import { createApp } from "vue";

import ListaLibros from "./componentes/ListaLibros.vue
import HolaMundo from "./componentes/HolaMundo.vue";
import router from "./router";

//createApp(HolaMundo).mount("#app");
//createApp(ListaLibros).mount("#app");
createApp(HolaMundo).use(router).mount("#app");
```

- Añadimos <router-view></router-view> en la app que carga la primera página:

resources > js > componentes > HolaMundo.vue

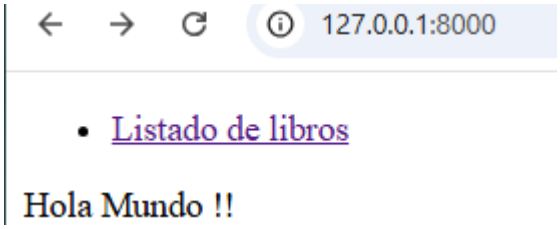
```
<template>
  <ul>
    <li>
      <router-link to="/libros">Listado de libros</router-link>
    </li>
  </ul>

  <div>
    <p>Hola Mundo !!</p>
  </div>

  <router-view></router-view>
</template>
```

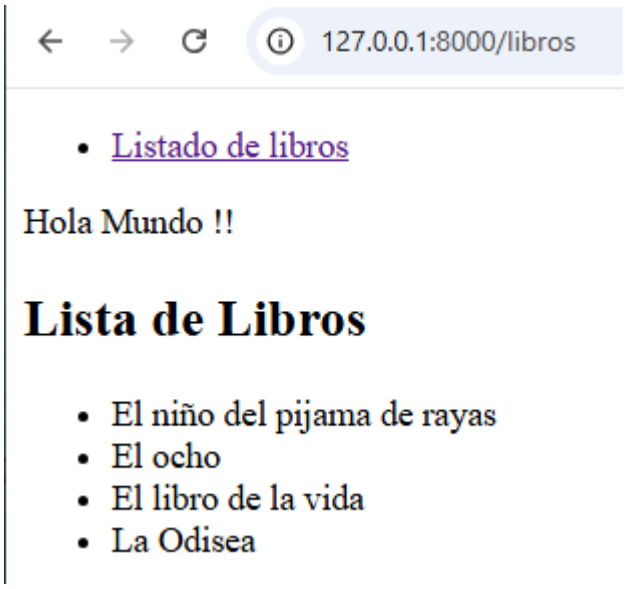
(Nota: Para que se muestren los estilos predefinidos del navegador y no se me reseteen todos los estilos debo comentar la primera línea del archivo resources>css>app.css)

resources > css > # app.css

```
/*@import 'tailwindcss';*/
```

- Ahora busco en el navegador la página de inicio y:




127.0.0.1:8000/libros
Listado de libros
Hola Mundo !!
Lista de Libros
El niño del pijama de rayas
El ocho
El libro de la vida
La Odisea